Linguaggi di Programmazione

Analisi sintattica: Linguaggi liberi e PDA

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Analisi Sintattica: Linguaggi Liberi e PDA

Linguaggi Liberi

- Classe di linguaggi più generale di quella dei linguaggi regolari: **linguaggi regolari $\subset$ linguaggi liberi.**
- Grammatiche libere da contesto più generali di quelle regolari:

Grammatiche regolari

$$A \to aB$$

$$A \to a$$

$$A \to \epsilon$$

$$A, B \in NT, a \in T$$

Grammatiche libere

$$A \to \alpha$$

$$\alpha \in (T \cup NT)^*$$

Automi a Pila (PDA)

Per riconoscere i linguaggi liberi dovremo usare una classe di automi più generale di quella usati per i linguaggi regolari (NFA-DFA).

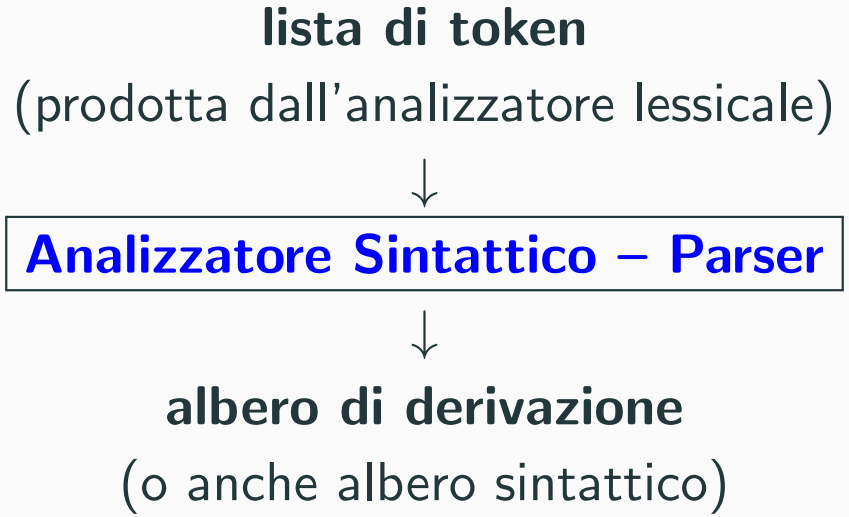
- Automi a pila (pushdown automata - PDA): automa a controllo finito che usa una memoria ausiliaria illimitata che funziona a pila (Last-in–First-out, LIFO).
  - PDA nondeterministici: dimostreremo che riconoscono tutti e soli i linguaggi liberi.
  - PDA deterministici: utili perché riconoscono i linguaggi liberi deterministici, quindi sicuramente non ambigui, quindi adatti per costruire compilatori.

Vedremo come ottenere un PDA a partire da una grammatica libera:

- con tecnica top-down: se la grammatica è di classe LL(k), allora il PDA funziona in modo deterministico;
- con tecnica bottom-up: se la grammatica è di classe LR(k), allora il PDA funziona in modo deterministico.

Analisi Sintattica

L'analizzatore sintattico (parser) genera, a partire dalla lista di token prodotta dall'analizzatore lessicale, un albero di derivazione (o anche un albero sintattico) relativo alla grammatica usata per costruirlo.
Infatti, il parser è prodotto a partire da una grammatica libera.



Breve ripasso su grammatiche libere

Grammatica libera da contesto
Una **grammatica libera da contesto** è una quadrupla (NT, T, R, S) dove:

- NT è un insieme finito di simboli nonterminali (di solito lettere maiuscole $A, B, ...$)
- T è un insieme finito di simboli terminali (di solito lettere minuscole $a, b, ...$)
- $S \in NT$ è il simbolo iniziale
- R è un insieme finito di produzioni (o regole) della forma $V \to \alpha$ dove $V \in NT$ e $\alpha \in (T \cup NT)^*$

Il linguaggio generato è $L(G) = \{w \in T^* | S \Rightarrow^* w\}$.
N.B.: se G è libera, allora L(G) è libero per definizione!

Breve ripasso su grammatiche libere (2)

Relazione $\Rightarrow$ di derivazione in un passo: (con $x, y, z \in (T \cup NT)^*$)

$$\frac{v = xAy \quad (A \to z) \in R \quad w = xzy}{v \Rightarrow w}$$

Relazione $\Rightarrow^*$ di derivazione in 0 o più passi (chiusura riflessiva e transitiva di $\Rightarrow$):

$$\frac{}{v \Rightarrow^* v} \qquad \frac{v \Rightarrow^* w \quad w \Rightarrow z}{v \Rightarrow^* z}$$

- **Derivazione canonica sinistra (leftmost)**: derivazione in cui, a partire da S, viene **sempre** riscritto il nonterminale più a sinistra.
- **Derivazione canonica destra (rightmost)**: derivazione in cui, a partire da S, viene **sempre** riscritto il nonterminale più a destra.
- Esiste una corrispondenza biunivoca tra derivazioni leftmost/rightmost ed alberi di derivazione.

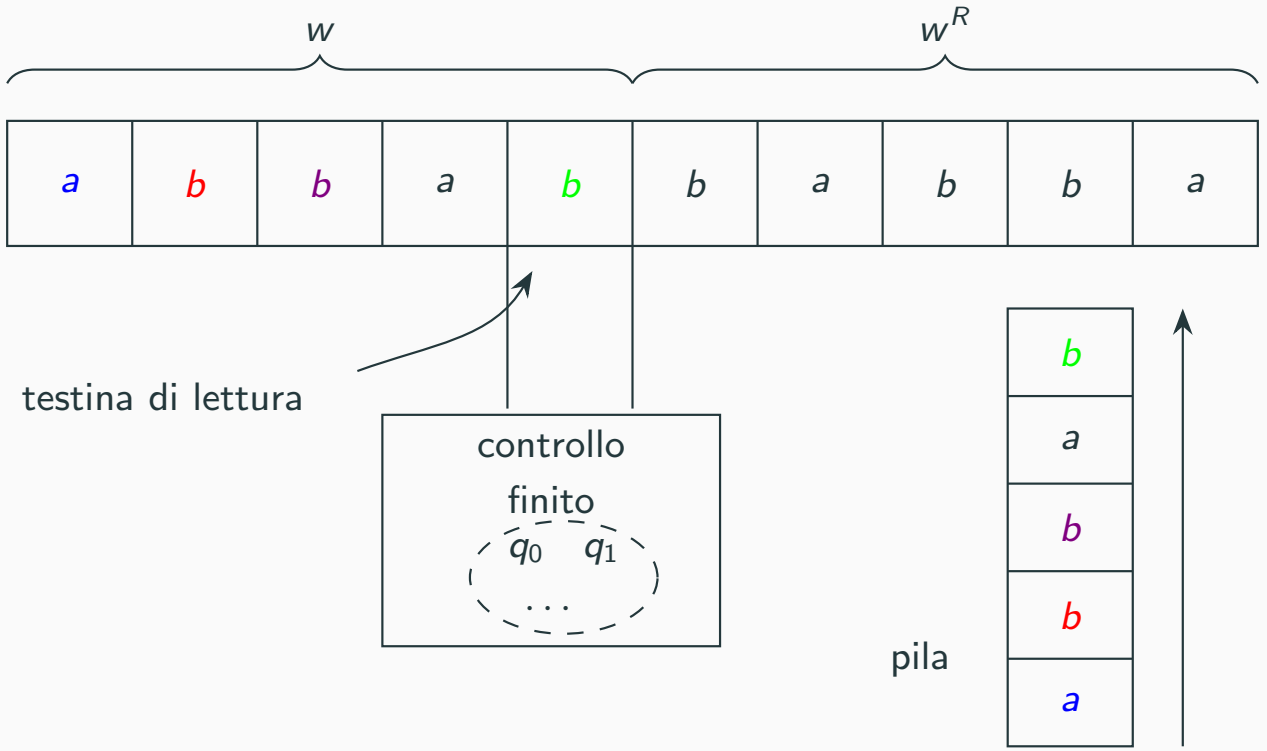
Come potenziare NFA/DFA per riconoscere linguaggi liberi?

Consideriamo, come esempio, $L = \{ww^R | w \in \{a, b\}^*\}$.

- L è libero perché $L = L(G)$ con $G = S \to \epsilon \mid aSa \mid bSb$ libera.
- L non è regolare (dimostrato in precedenza usando il pumping lemma a rovescio).
- $L = \{\epsilon, aa, bb, aaaa, abba, baab, bbbb, ...\}$

Per riconoscere una stringa della forma ww^R, l'automa deve "ricordare" la prima metà dell'input (cioè w) in modo da confrontarla con la seconda metà (cioè w^R) in ordine inverso: necessità di **accumulare** (parte del)l' input!
Bisogna fornire all'automa una memoria ausiliaria, una **pila**, che può crescere illimitatamente (perché w può essere lunga a piacere).
Quando si accorge che w è finita (ovvero quando si accorge di essere a metà della stringa in input)?
In genere, non c'è nessun criterio: nondeterministicamente l'automa assume di essere arrivato al centro/metà ed inizia da lì in poi a confrontare all'indietro!

Come potenziare NFA/DFA per riconoscere linguaggi liberi?



- la pila può crescere senza limiti (memoria ausiliaria illimitata, perché w può essere arbitrariamente lunga);
- ma può leggere solo l'elemento top della pila;
- può rimuovere solo l'elemento top della pila;
- può inserire solo in testa;
- ovvero, disciplina LIFO (Last-in–First-out).

Automa a pila nondeterministico (PDA)

Automa a pila nondeterministico
Un PDA è una 7-upla $(\Sigma, Q, \Gamma, \delta, q_0, \bot, F)$ dove

- Σ è un alfabeto finito di simboli in input;
- Q è un insieme finito di stati;
- Γ è un insieme finito di simboli della pila;
- δ è la funzione di transizione con tipo
$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \to \mathcal{P}_{fin}(Q \times \Gamma^*)$$
- $q_o \in Q$ è lo stato iniziale;
- $\bot \in \Gamma$ è il simbolo iniziale sulla pila;
- $F \subseteq Q$ è l'insieme di stati finali.

Funzione δ e nondeterminismo

La **funzione** $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \to \mathcal{P}_{fin}(Q \times \Gamma^*)$ specifica che:

- dato lo stato corrente (elemento di Q),
- dato un simbolo in lettura (elemento di Σ), oppure senza leggere niente (caso ϵ),
- dato il simbolo in cima alla pila (elemento di Γ), che viene rimosso;
- restituisce un insieme finito di coppie (q, γ) (cioè un elemento di $\mathcal{P}_{fin}(Q \times \Gamma^*)$), dove q è uno stato e $\gamma \in \Gamma^*$ è la stringa (eventualmente vuota) che va messa in cima alla pila al posto del simbolo "rimosso".

IL PDA è **nondeterministico** perché

- $|\delta(q, \sigma, A)|$ può essere > 1 (per qualche $q \in Q, \sigma \in \Sigma \cup \{\epsilon\}, A \in \Gamma$).
  - Ma anche se richiedessimo che $|\delta(q, \sigma, A)| \leq 1$ sia sempre soddisfatta per ogni $q \in Q, \sigma \in \Sigma \cup \{\epsilon\}, A \in \Gamma$, potrebbero esistere $q' \in Q, b \in \Sigma, B \in \Gamma$ tali che $|\delta(q', \epsilon, B)| = 1$ e $|\delta(q', b, B)| = 1$, causando nondeterminismo tra mossa ϵ e mossa b.

Transizioni di un PDA

- Descrizione istantanea (o configurazione): (q, w, β) dove
  - $q \in Q$ (stato corrente)
  - $w \in \Sigma^*$ (input ancora non letto)
  - $\beta \in \Gamma^*$ (stringa sulla pila, il top è il simbolo più a sinistra).
- Mossa: dati $a \in \Sigma$, $X \in \Gamma$, $\alpha, \beta \in \Gamma^*$

$$\frac{(q', \alpha) \in \delta(q, a, X)}{(q, aw, X\beta) \vdash_N (q', w, \alpha\beta)} \quad (1)$$

$$\frac{(q', \alpha) \in \delta(q, \epsilon, X)}{(q, w, X\beta) \vdash_N (q', w, \alpha\beta)} \quad (2)$$

- Computazione/cammino: la chiusura riflessiva e transitiva di $\vdash_N$, indicata con $\vdash_N^*$

$$\frac{}{(q, w, \beta) \vdash_N^* (q, w, \beta)} \qquad \frac{(q, w, \beta) \vdash_N^* (q', w', \beta') \vdash_N (q'', w'', \beta'')}{(q, w, \beta) \vdash_N^* (q'', w'', \beta'')}$$

Linguaggio accettato da un PDA

Due modalità di riconoscimento:

- **per stato finale**:

$$L[N] = \{w \in \Sigma^* | (q_0, w, \perp) \vdash_N^* (q, \epsilon, \alpha) \text{ con } q \in F\}$$

- **per pila vuota**:

$$P[N] = \{w \in \Sigma^* | (q_0, w, \perp) \vdash_N^* (q, \epsilon, \epsilon)\}$$

Osservazione: Per un certo PDA N, spesso $L[N] \neq P[N]$.
Osservazione: Tuttavia, dimostreremo che se $L = L[N]$, allora esiste N' tale che $L = P[N']$, e viceversa. Cioè la classe dei linguaggi riconosciuti dai PDA per stato finale è la stessa di quella riconosciuta dai PDA per pila vuota.

Esempio: PDA per $\{ww^R | w \in \{a, b\}^*\}$

$N = (\Sigma, Q, \Gamma, \delta, q_0, \perp, F)$ dove

- $\Sigma = \{a, b\}$
- $Q = \{q_0, q_1, q_2\}$
- $\Gamma = \{a, b, \perp\}$
- $F = \{q_2\}$
- δ è definita come:

$$\delta(q_0, a, X) = \{(q_0, aX)\} \quad \forall X \in \Gamma \qquad \text{impilo}$$

$$\delta(q_0, b, X) = \{(q_0, bX)\} \quad \forall X \in \Gamma \qquad \text{impilo}$$

$$\delta(q_0, \epsilon, X) = \{(q_1, X)\} \quad \forall X \in \Gamma \qquad \text{passo nondet.}$$

$$\delta(q_1, a, a) = \{(q_1, \epsilon)\} \qquad \text{faccio match}$$

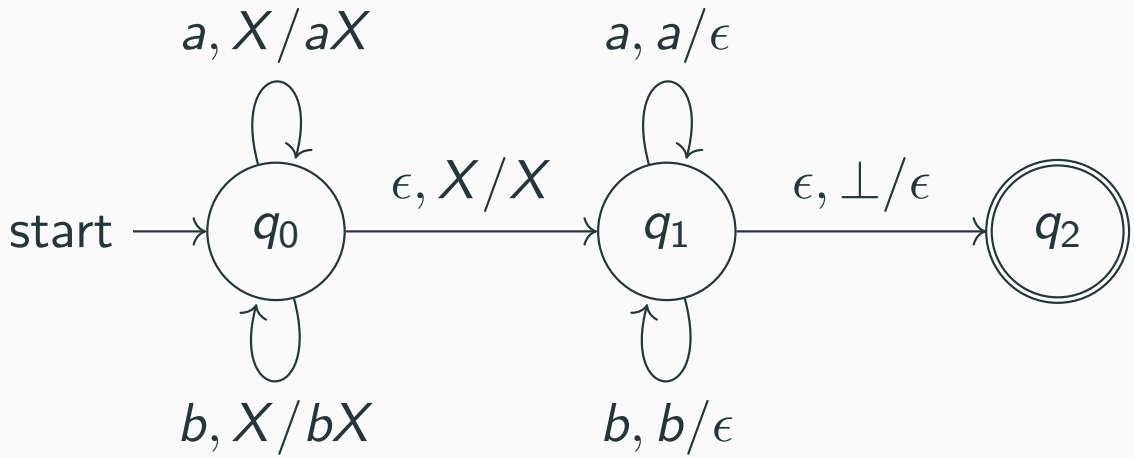
$$\delta(q_1, b, b) = \{(q_1, \epsilon)\} \qquad \text{faccio match}$$

$$\delta(q_1, \epsilon, \perp) = \{(q_2, \epsilon)\} \qquad \text{accept}$$

$$\delta(q_2, \sigma, X) = \emptyset \quad \forall \sigma \in (\Sigma \cup \{\epsilon\}), \forall X \in \Gamma$$

Esempio: PDA per $\{ww^R | w \in \{a, b\}^*\}$

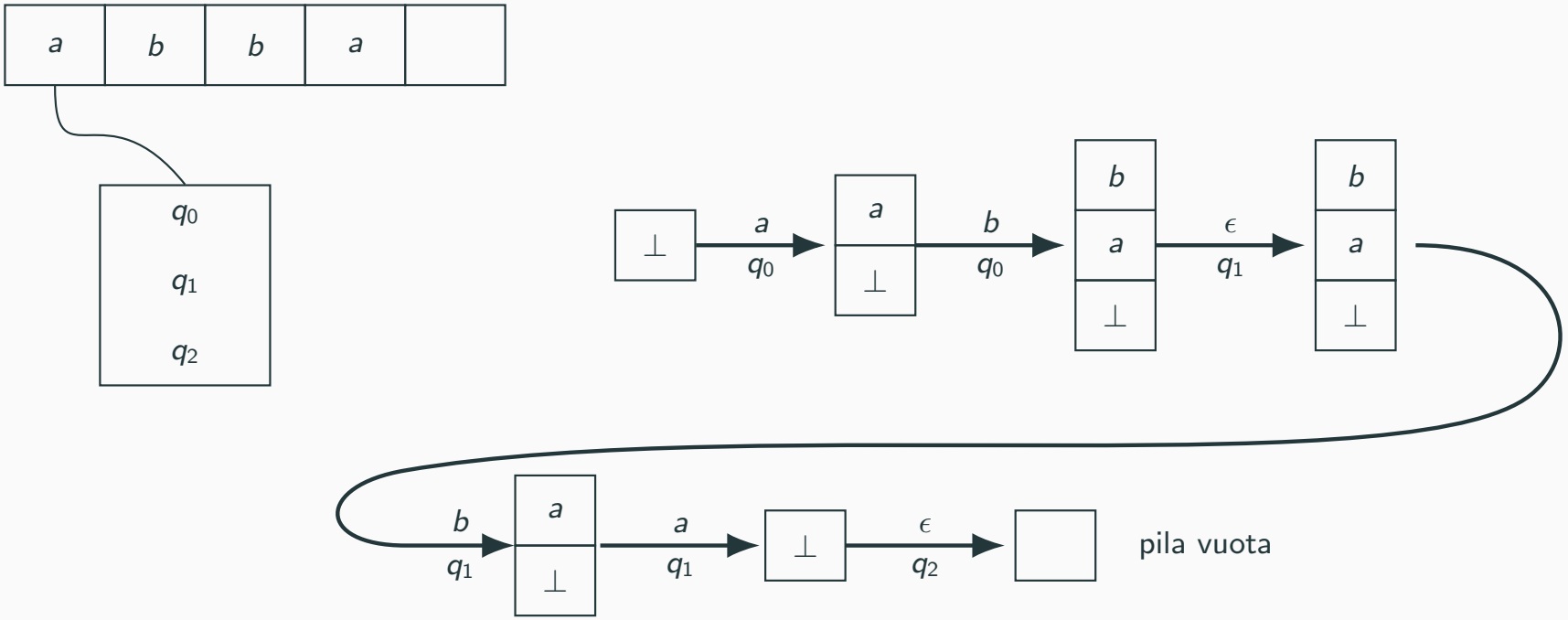
Diagramma di transizione per N:



La transizione $q_0 \xrightarrow{\epsilon, X/X} q_1$ è un passo nondeterministico: l'automa decide con questa mossa di passare dalla modalità "impilo" alla modalità "faccio match".

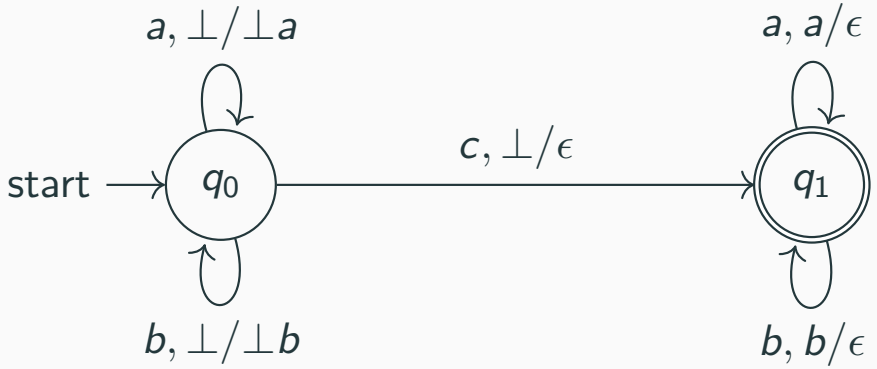
La transizione $q_1 \xrightarrow{\epsilon, \perp/\epsilon} q_2$ è un passo deterministico: l'automa riconosce che ha svuotato la pila e quindi accetta la stringa che ha letto fino a quel momento (N.B.: potrebbe non aver finito di leggere l'input, ma quello che ha letto è una stringa corretta della forma ww^R).

Esempio: PDA per $\{ww^R | w \in \{a, b\}^*\}$



- esiste una "computazione" per *abba* che porta l'automa N allo stato finale q_2 (così come a svuotare la pila), quindi *abba* è riconosciuta (sia per stato finale che per pila vuota).
- si può vedere che "svuoto la pila se e solo se vado sullo stato q_2", quindi $L[N] = P[N]$!

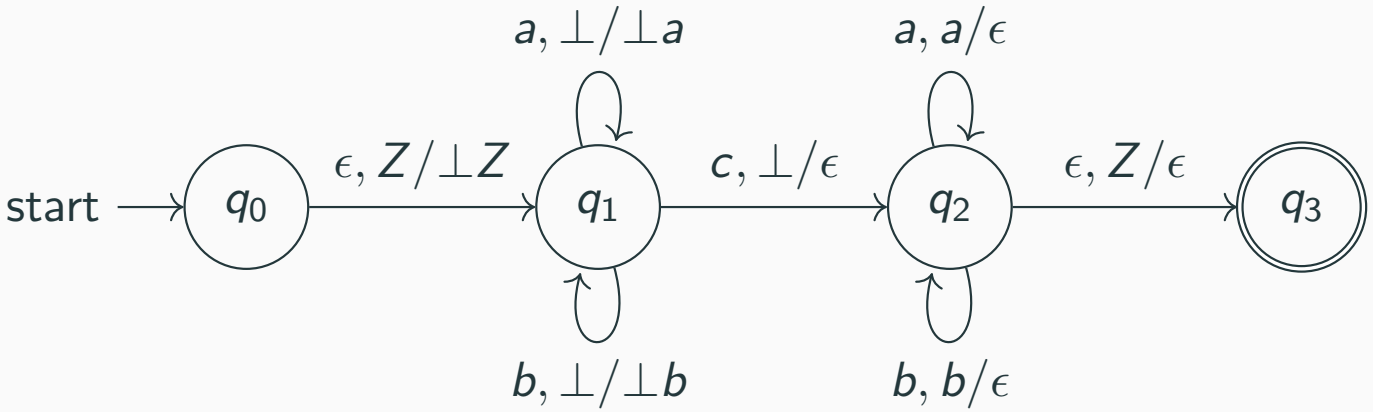
Esempio: PDA per $L = \{wcw^R | w \in \{a, b\}^*\}$



- Il simbolo $\perp$ galleggia in cima allo stack finché impilo. Poi viene rimosso quando leggo il separatore c.
- N è deterministico: vale sempre $|\delta(q, a, X)| \leq 1$ per ogni $a \in \Sigma, X \in \Gamma, q \in Q$ e non ci sono mosse ϵ.
- Diversamente da un DFA, un PDA deterministico non garantisce di leggere tutto l'input; ad esempio, N si blocca con input acc.
- $L = P[N]$ cioè il linguaggio L è riconosciuto per pila vuota.
- $L[N] = \{wcy | w \in \{a, b\}^* \wedge y$ è prefisso di $w^R\}$; ad esempio, $ac \in L[N]$. Quindi $L[N] \neq P[N]$.
- Se l'input è $acab$, questo non viene letto tutto: N si blocca dopo aver letto aca sullo stato finale, cioè riconosce aca ma non $acab$.

Esempio: altro PDA per $L = \{wcw^R | w \in \{a, b\}^*\}$

Come modificare il PDA deterministico della pagina precedente per far sì che riconosca L per stato finale?



- Introduco un nuovo simbolo iniziale Z che rimane in fondo alla pila.
  - Quando, nello stato q_2, riappare Z, vuol dire che la pila "originale" è stata svuotata, e quindi posso ora svuotarla del tutto, cancellando Z e raggiungendo lo stato finale q_3.
  - N è deterministico e riconosce L sia per stato finale, sia per pila vuota: $L = L[N] = P[N]$.
- Vediamo ora che questa tecnica è generale, ovvero: la classe dei linguaggi riconosciuti per pila vuota o per stato finale non cambia.

Teorema: stessa classe per le due modalità di accettazione

Teorema

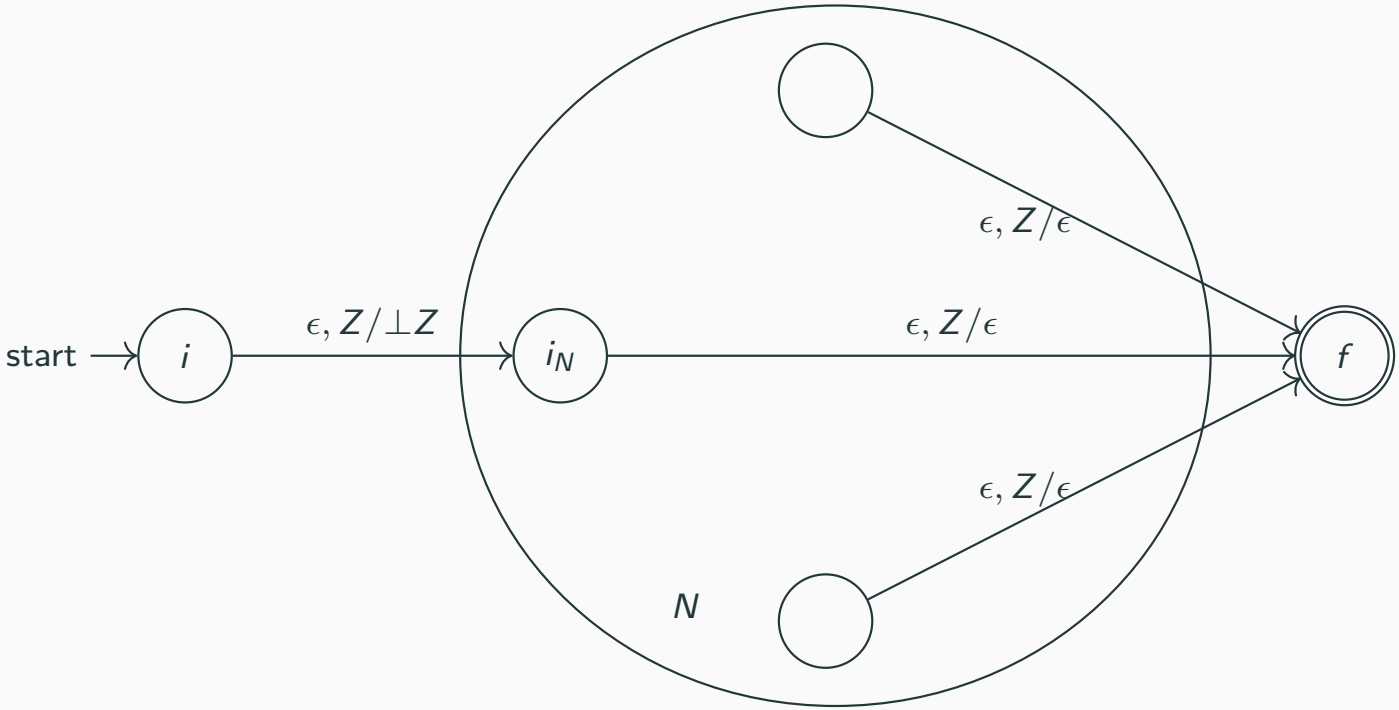
1. Dato un PDA N, se $L = P[N]$ (per pila vuota), possiamo costruire un PDA N' tale che $L = L[N']$.
2. Dato un PDA N, se $L = L[N]$ (per stato finale), possiamo costruire un PDA N' tale che $L = P[N']$.

Ovvero la classe dei linguaggi riconosciuti dai PDA per pila vuota è la stessa di quella che viene riconosciuta dai PDA per stato finale.

Dimostrazione di (1)

Dato N che riconosce per pila vuota (con $\perp$ come simbolo iniziale sulla pila), costruiamo N' che riconosce per stato finale (con Z come simbolo iniziale sulla pila):

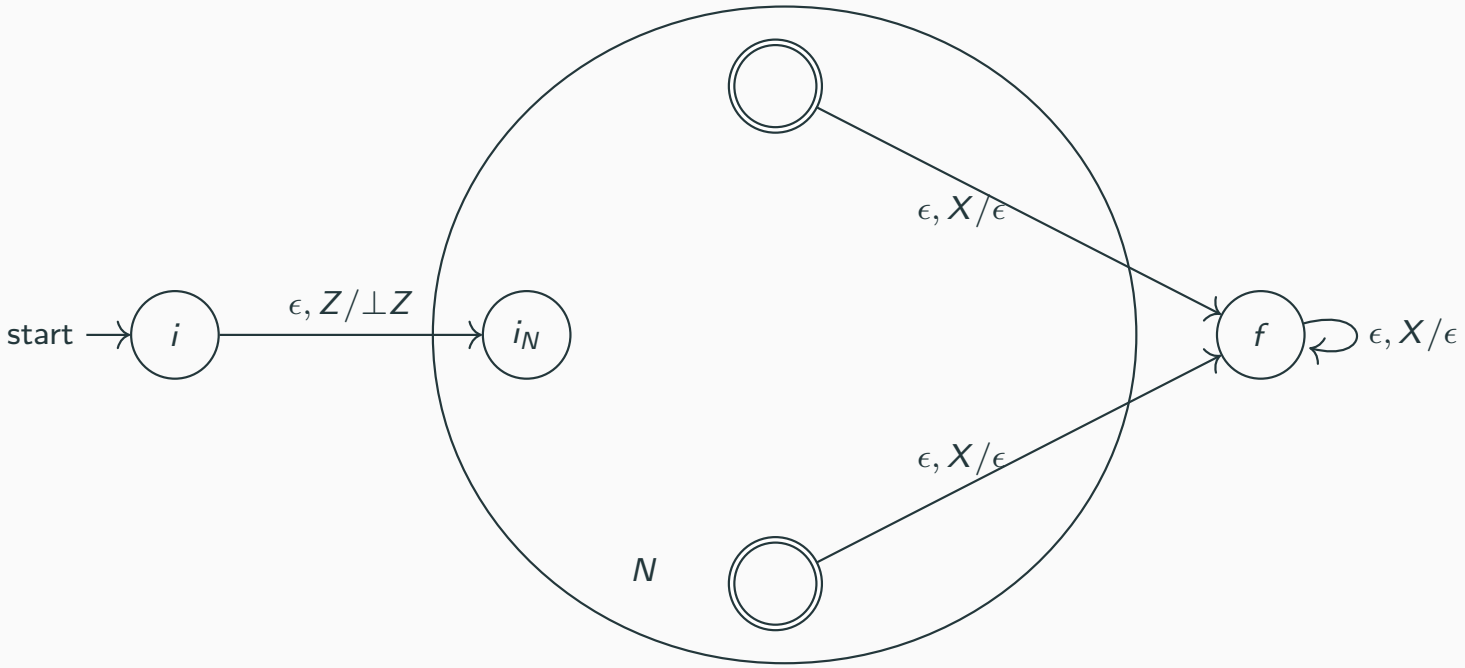
- Aggiungo un nuovo stato iniziale i e un nuovo stato finale f.
- Gli eventuali stati finali di N non sono più finali per N'.
- Aggiungo un arco, etichettato ϵ, da i a i_N che introduce un nuovo simbolo Z di fine pila, che rimane costantemente in fondo alla pila.
- Aggiungo da ogni stato di N un arco verso f in modo che se la pila "originale" risultasse vuota, con quella mossa, che cancella Z, la rende realmente vuota.



Dimostrazione di (2)

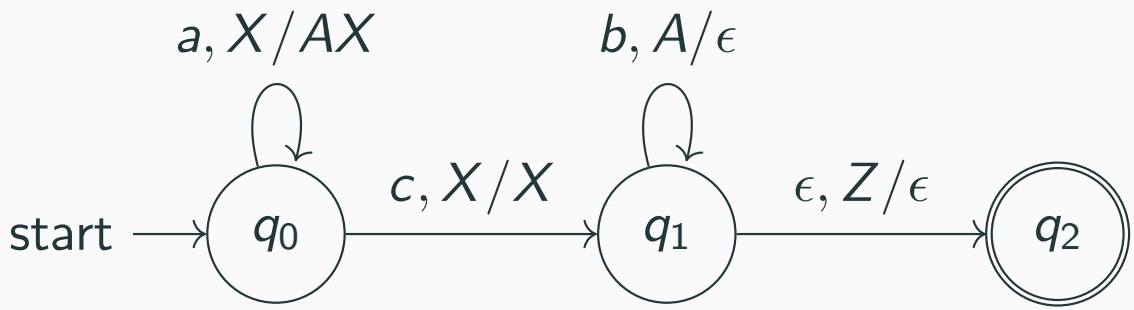
Dato N che riconosce per stato finale (con $\perp$ come simbolo iniziale sulla pila), costruiamo N' che riconosce per pila vuota (con Z come simbolo iniziale sulla pila):

- Aggiungo un nuovo stato iniziale i e un nuovo stato f.
- Aggiungo un arco, etichettato ϵ, da i a i_N che introduce un nuovo simbolo Z di fine pila, che rimane costantemente in fondo alla pila, per evitare che uno stato di N possa svuotare la pila.
- Aggiungo da ogni stato finale di N un arco verso lo stato f che elimina un simbolo dalla pila (che non può essere vuota, visto che almeno Z ci sarà); lo stato f può continuare a fare pop dalla pila, fino a svuotarla.



Esempio: dato un linguaggio, costruire un PDA

Prendiamo $L = \{a^n c b^n | n \geq 0\}$.
Costruiamo un PDA che usa Z come simbolo iniziale, $\Gamma = \{A, Z\}$ e usiamo X per indicare un qualunque elemento di Γ.
Un possibile diagramma di transizione è il seguente:

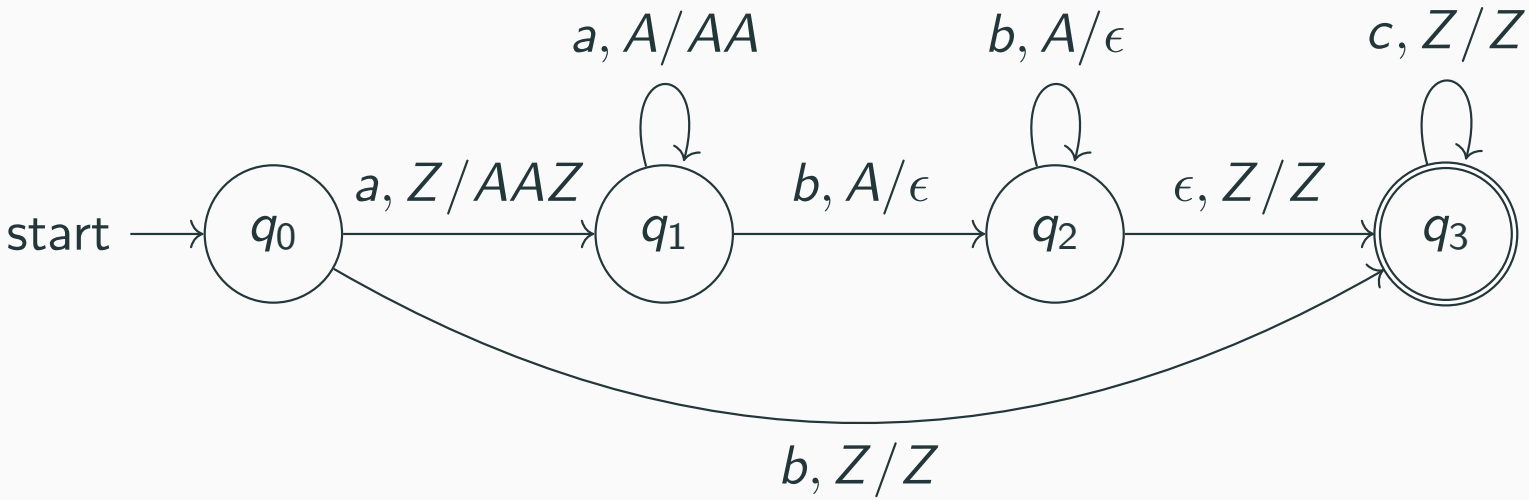


$$L = L[N] = P[N]$$

In questa soluzione, il simbolo di fine pila Z rimane sempre in fondo alla pila, e viene rimosso solo nell'ultima transizione.

Esempio (2)

Consideriamo $L = \{a^n b^{n+1} c^m | n, m \geq 0\}$.

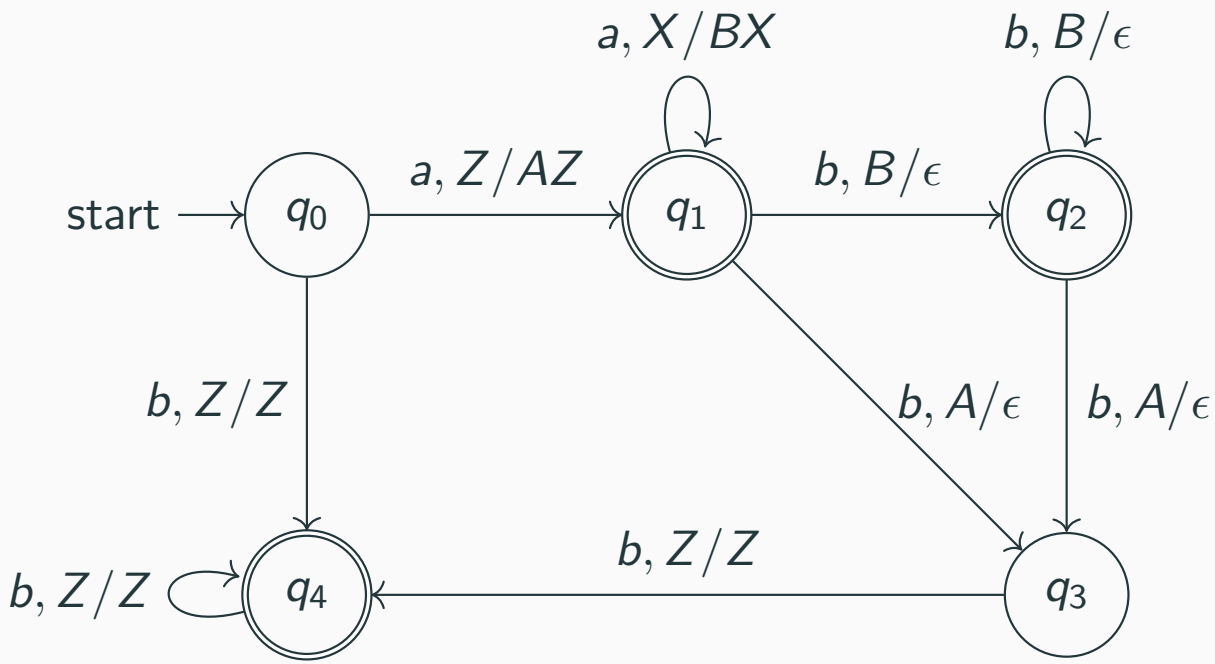


$L = L[N]$ (per stato finale), $P[N] = \emptyset$

Esercizio: Modificare questo PDA affinché riconosca L per pila vuota.

Esempio (3)

Consideriamo $L = \{a^n b^m | n \neq m \wedge n, m \geq 0\}$ e costruiamo un PDA che riconosce L per stato finale.



Si arriva sullo stato q_3 solo se le b, che ho letto dopo le a, sono tante quante le a; quindi q_3 è non finale.
Anche q_0 è non finale, perché $\epsilon = a^0 b^0 \notin L$.

Come ottenere un PDA da una grammatica libera?

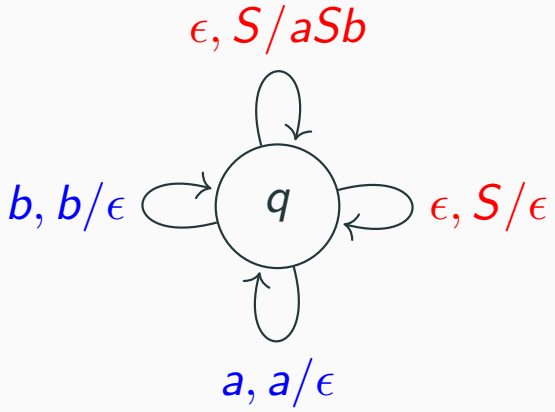
Un possibile metodo (top-down) è illustrato dal seguente esempio.

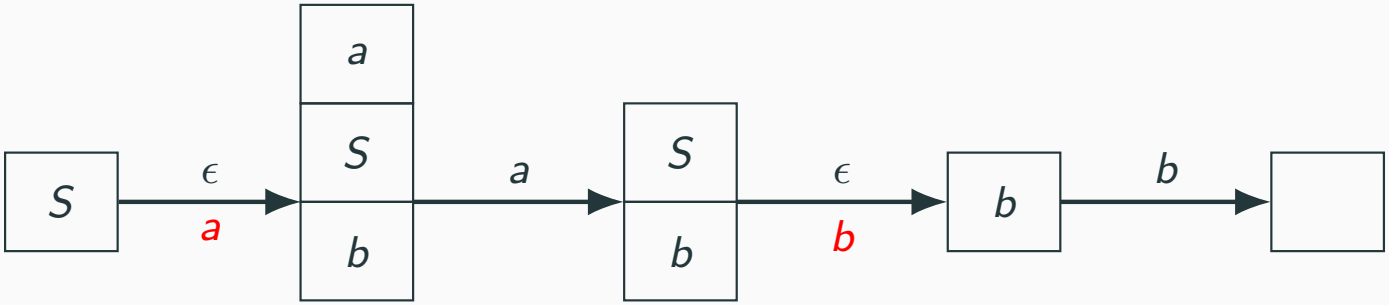
- Prendiamo $G = S \to aSb \mid \epsilon$.
- Costruiamo un PDA con un solo stato tale che:
  - il simbolo inizialmente sulla pila sia S;
  - riconosca $L = \{a^n b^n \mid n \geq 0\}$ per pila vuota;
- Ci sono due tipi di transizioni:
  - espandi: transizione ϵ che sostituisce S in cima alla pila con la parte destra di una sua produzione;
  - consuma: transizione eseguibile se viene fatto il match tra il prossimo input e il simbolo in cima alla pila.

Il PDA risultante è nondeterministico perché la scelta della transizione di tipo espandi (quando S è e in cima alla pila) è completamente libera.

Come ottenere un PDA da una grammatica libera? (2)

$G = S \to aSb \mid \epsilon$



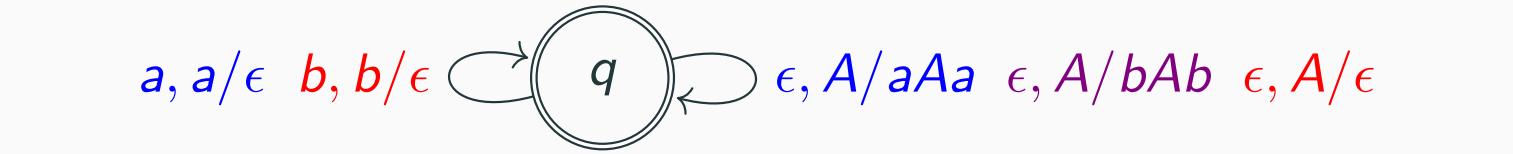


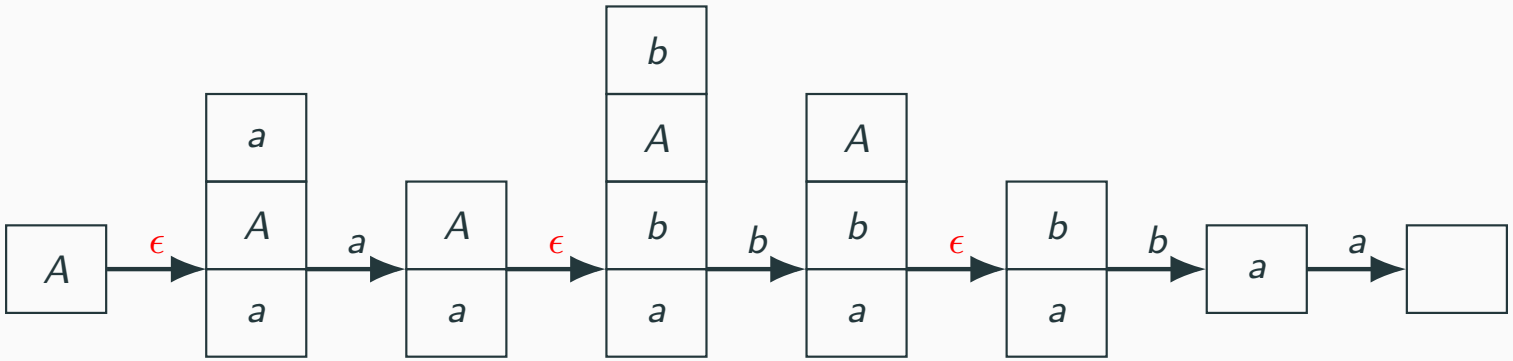
La stringa ab viene riconosciuta per pila vuota e origina la derivazione $S \Rightarrow aSb \Rightarrow ab$.

Posso scegliere la transizione di tipo espandi guardando in anticipo quale sia il prossimo simbolo in input (lookahead). Posso così risolvere il nondeterminismo (per questa grammatica).

Come ottenere un PDA da una grammatica libera? (3)

Sia $G = A \to aAa \mid bAb \mid \epsilon$. Il linguaggio $L(G) = \{ww^R \mid w \in \{a, b\}^*\}$. Costruiamo il PDA con un solo stato, con A sulla pila come simbolo iniziale, che riconosce per pila vuota.





Costruisco l'associata derivazione leftmost per *abba*, data dalle 3 transizioni ϵ di tipo espandi :

$$A \Rightarrow aAa \Rightarrow abAba \Rightarrow abba$$

Esercizio: Perché non riesco a risolvere il nondeterminismo col lookahead?

Esercizio: Ricava l'albero di derivazione da questa derivazione leftmost.

Teorema: i PDA riconoscono tutti e soli i linguaggi liberi

Teorema
Un linguaggio L è libero da contesto se e solo se è accettato da un PDA.

Proviamo separatamente le due implicazioni:

1. Se L è libero, allora $\exists N$ PDA tale che $L = P[N]$ (riconoscimento per pila vuota).
2. Se $L = P[N]$, allora esiste G libera tale che $L = L(G)$.

Caso (1): Se L è libero, allora $\exists N$ PDA tale che $L = P[N]$.

Se L è libero, allora esiste $G = (NT, T, R, S)$ libera tale che $L = L(G)$. Costruiamo il PDA $N = (T, \{q\}, T \cup NT, \delta, q, S, \emptyset)$ dove la funzione di transizione δ è definita come segue:

$$\forall A \in NT: \quad \delta(q, \epsilon, A) = \{(q, \beta) | A \to \beta \in R\} \qquad \text{espandi}$$

$$\forall a \in T: \quad \delta(q, a, a) = \{(q, \epsilon)\} \qquad \text{consuma}$$

Ogni volta che N ha un nonterminale A in cima alla pila, sceglie nondeterministicamente una produzione per A, senza consumare l'input, e rimpiazza A con la parte destra della produzione. Poiché il nonterminale in cima alla pila è quello più a sinistra in una derivazione, allora N simula la costruzione di una derivazione canonica sinistra.
Se invece sulla pila c'è un terminale a e se l'input presenta proprio a, questi vengono consumati. Se l'input è $b \neq a$, ci si blocca, e si fa backtracking, provando con un'altra produzione.
Si può dimostrare, per induzione sulla lunghezza di w, che

$$S \Rightarrow^*_{leftmost} w \text{ se e solo se } (q, w, S) \vdash^*_N (q, \epsilon, \epsilon)$$

Caso (2): Se $L = P[N]$, allora esiste G libera tale che $L = L(G)$.

Molto più difficile: prova solo schematizzata.

- **Lemma 1**: ogni PDA N, che riconosce per pila vuota, può essere simulato da un PDA N' con un solo stato (aumentando opportunamente i simboli della pila).
- **Lemma 2**: ogni PDA con un solo stato ha un'equivalente grammatica libera.
  - **Idea della prova**:
    Se $(q, B_1 B_2 ... B_k) \in \delta(q, a, A)$, con $a \in \Sigma \cup \{\epsilon\}$, nel PDA con un solo stato, allora la grammatica G contiene la produzione

$$A \to aB_1 B_2 ... B_k$$

    Si può dimostrare che

$$S \Rightarrow^*_{leftmost} w \text{ se e solo se } (q, w, S) \vdash^*_N (q, \epsilon, \epsilon)$$

c.v.d